

SEQUENCE & TIME-SERIES PATTERNS

Senior Developer Mastery Guide

14 Core Patterns · Window Functions · Gap & Island Detection

Calendar Tables · Moving Aggregates · Forecasting in SQL

35 Graded Exercises · Full Answer Key

Everything a senior developer needs to master time in SQL

Table of Contents

FOUNDATIONS

- Ch 1** Mental Model — Time, Sequences & SQL
- Ch 2** Schemas & Sample Data Used Throughout

SEQUENCE PATTERNS

- Ch 3** Pattern 1 — Generating Number & Date Sequences
- Ch 4** Pattern 2 — Running Totals & Cumulative Aggregates
- Ch 5** Pattern 3 — Lag, Lead & Row-Over-Row Comparisons
- Ch 6** Pattern 4 — Moving Averages & Rolling Windows
- Ch 7** Pattern 5 — Ranking & Percentile Functions
- Ch 8** Pattern 6 — Nth Value, First & Last in a Window

TIME-SERIES PATTERNS

- Ch 9** Pattern 7 — Gap Detection & Missing Periods
- Ch 10** Pattern 8 — Island Detection (Consecutive Groups)
- Ch 11** Pattern 9 — Session & Event Windowing
- Ch 12** Pattern 10 — Period Overlap & Allen's Interval Algebra
- Ch 13** Pattern 11 — Calendar Tables & Business Day Arithmetic
- Ch 14** Pattern 12 — Time-Weighted Averages
- Ch 15** Pattern 13 — Forecasting & Linear Regression in SQL
- Ch 16** Pattern 14 — Slowly Changing Dimensions (SCD Type 2)

MASTERY

- Ch 17** Performance, Indexing & Anti-Patterns
- Ch 18** Dialect Reference: PostgreSQL / MySQL / SQL Server / Oracle
- Ch 19** 35 Graded Exercises
- Ch 20** Full Answer Key

Chapter 1

Mental Model — Time, Sequences & SQL

Why Time Is Hard in SQL

SQL is a set-based language. A `SELECT` returns an unordered bag of rows. Time is inherently ordered and sequential — exactly the opposite of what SQL naturally models. Every pattern in this guide is a technique for imposing order and sequence on fundamentally orderless relational data.

■ The Three Axes of Time Problems in SQL

1. **SEQUENCE** — 'What happened before/after this row?' → Window functions, `LAG/LEAD`
2. **PERIOD** — 'What was true during this interval?' → Allen's algebra, SCD Type 2
3. **CALENDAR** — 'What are the business days between dates?' → Calendar tables, CTE generators

Core Vocabulary

- **Window Function** — A function that computes a result across a set of rows related to the current row, without collapsing them. The frame is defined by `OVER (PARTITION BY ... ORDER BY ...)`.
- **Frame Clause** — `ROWS / RANGE BETWEEN X PRECEDING AND Y FOLLOWING` — defines which rows relative to the current row are included in the window.
- **Partition** — `PARTITION BY` logically splits the dataset into independent groups. The window function restarts at each partition boundary.
- **Gap** — A missing period in a time series — a date/timestamp range where no row exists but one was expected.
- **Island** — A maximal consecutive sequence of rows that belong together (e.g. consecutive days with activity).
- **SCD Type 2** — Slowly Changing Dimension Type 2 — stores history by adding new rows with `valid_from / valid_to` columns rather than overwriting.
- **Allen's Intervals** — A formal algebra of 13 relations between two time intervals (before, meets, overlaps, during, starts, finishes, equals, and their inverses).
- **Time-Weighted Avg** — An average weighted by the duration of each value's validity — used in finance, sensor data, and inventory valuation.

- **Running Total** — A cumulative sum of values from the first row to the current row, within a partition.
- **Moving Average** — An average computed over a fixed-size sliding window of rows, smoothing out short-term fluctuations.

The Window Function Anatomy

Window Function — full anatomy

```
SELECT
col,
SUM(amount) OVER ( -- aggregate function
PARTITION BY dept -- restart per dept
ORDER BY sale_date -- define row order
ROWS BETWEEN UNBOUNDED PRECEDING -- frame start
AND CURRENT ROW -- frame end
) AS running_total
FROM sales;
```

Clause	Purpose	Example Values
PARTITION BY	Splits into independent groups	dept, user_id, sensor_id
ORDER BY	Defines row sequence within partition	sale_date ASC, event_time ASC
ROWS BETWEEN	Physical row-based frame	3 PRECEDING AND CURRENT ROW
RANGE BETWEEN	Value-based frame (same ORDER BY value = same frame)	INTERVAL '7 days' PRECEDING
GROUPS BETWEEN	Peer-group-based frame (PostgreSQL 11+)	1 PRECEDING AND 1 FOLLOWING

Chapter 2

Schemas & Sample Data Used Throughout

All schemas used in this guide

```
-- Primary schema: sales transactions

CREATE TABLE sales (
  sale_id INT PRIMARY KEY,
  emp_id INT NOT NULL,
  dept VARCHAR(50),
  amount DECIMAL(12,2),
  sale_date DATE NOT NULL
);

-- Sensor readings (time-series)

CREATE TABLE sensor_readings (
  sensor_id INT,
  recorded_at TIMESTAMP NOT NULL,
  value DECIMAL(10,3),
  PRIMARY KEY (sensor_id, recorded_at)
);

-- Employee SCD Type 2 history

CREATE TABLE emp_history (
  hist_id INT PRIMARY KEY,
  emp_id INT NOT NULL,
  dept VARCHAR(50),
  salary DECIMAL(10,2),
  valid_from DATE NOT NULL,
  valid_to DATE, -- NULL means current record
  is_current BOOLEAN DEFAULT TRUE
);

-- Bookings / events with time periods

CREATE TABLE bookings (
```

```
booking_id INT PRIMARY KEY,  
resource VARCHAR(50),  
start_ts TIMESTAMP NOT NULL,  
end_ts TIMESTAMP NOT NULL  
);
```

Table	Primary Use	Key Columns
sales	Running totals, ranking, lag/lead	emp_id, amount, sale_date
sensor_readings	Gap detection, islands, time-weighted avg	sensor_id, recorded_at, value
emp_history	SCD Type 2, period queries	emp_id, valid_from, valid_to
bookings	Overlap detection, session windows	resource, start_ts, end_ts

Chapter 3

Pattern 1 — Generating Number & Date Sequences

Why You Need a Sequence Generator

Most time-series analysis requires a complete, unbroken list of dates or numbers to LEFT JOIN against real data. Without it, gaps simply disappear — missing data looks like zero data. A sequence generator is the foundation of correct time-series work.

3.1 PostgreSQL — generate_series() (Fastest)

PostgreSQL — generate_series()

```
-- Integer sequence 1..100

SELECT generate_series(1, 100) AS n;
```

```
-- Date series: every day in 2024
```

```
SELECT generate_series(
  '2024-01-01'::DATE,
  '2024-12-31'::DATE,
  INTERVAL '1 day'
)::DATE AS dt;
```

```
-- Hourly timestamps for a week
```

```
SELECT generate_series(
  '2024-06-01 00:00'::TIMESTAMP,
  '2024-06-07 23:00'::TIMESTAMP,
  INTERVAL '1 hour'
) AS ts;
```

```
-- Monthly sequence
```

```
SELECT generate_series(
  '2024-01-01'::DATE,
  '2024-12-01'::DATE,
  INTERVAL '1 month'
)::DATE AS month_start;
```

3.2 Portable Recursive CTE Sequence

Portable recursive date generator

```
-- Works in PostgreSQL, MySQL 8+, SQL Server, Oracle

WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 365
)

SELECT
  n,
  CAST('2024-01-01' AS DATE) + (n - 1) * INTERVAL '1 day' AS dt
FROM seq;
```

3.3 Gap-Filling — LEFT JOIN Date Series to Real Data

Gap-fill: complete date series with zero-fill

```
-- Show every day in Jan 2024, even those with no sales

WITH RECURSIVE cal AS (
  SELECT CAST('2024-01-01' AS DATE) AS dt
  UNION ALL
  SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < '2024-01-31'
)

SELECT
  cal.dt,
  COALESCE(SUM(s.amount), 0) AS daily_revenue,
  COALESCE(COUNT(s.sale_id), 0) AS sale_count
FROM cal
LEFT JOIN sales s ON s.sale_date = cal.dt
GROUP BY cal.dt
ORDER BY cal.dt;
```

3.4 Cross-Partition Gap Fill (Per Employee)

Cross-partition: employee × day matrix with zeros

```
-- Every employee, every day – whether they sold or not
```



```
WITH RECURSIVE cal AS (  
  SELECT CAST('2024-01-01' AS DATE) AS dt  
  UNION ALL  
  SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < '2024-01-31'  
)  
emps AS (SELECT DISTINCT emp_id FROM sales)  
SELECT  
  e.emp_id,  
  c.dt,  
  COALESCE(SUM(s.amount), 0) AS daily_amount  
FROM cal c  
CROSS JOIN emps e  
LEFT JOIN sales s ON s.sale_date = c.dt AND s.emp_id = e.emp_id  
GROUP BY e.emp_id, c.dt  
ORDER BY e.emp_id, c.dt;
```

◆ TIP

Always LEFT JOIN your real data onto the calendar, not the reverse. If you join the calendar onto real data, days with no activity disappear.

Chapter 4

Pattern 2 — Running Totals & Cumulative Aggregates

4.1 Basic Running Total

Running total across all rows

```
SELECT
  sale_id,
  emp_id,
  sale_date,
  amount,
  SUM(amount) OVER (ORDER BY sale_date, sale_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total,
  COUNT(*) OVER (ORDER BY sale_date, sale_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_count
FROM sales
ORDER BY sale_date, sale_id;
```

4.2 Running Total Partitioned by Employee

Running total per employee with % progress

```
SELECT
  emp_id,
  sale_date,
  amount,
  SUM(amount) OVER (
    PARTITION BY emp_id
    ORDER BY sale_date, sale_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS emp_running_total,
  -- Running total as % of employee's grand total
```

```
ROUND(
100.0 * SUM(amount) OVER (
PARTITION BY emp_id
ORDER BY sale_date, sale_id
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) / SUM(amount) OVER (PARTITION BY emp_id),
2) AS pct_of_emp_total
FROM sales
ORDER BY emp_id, sale_date;
```

4.3 Running Min, Max, Average

Running min / max / avg / stddev

```
SELECT
sale_date,
amount,
MIN(amount) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS running_min,
MAX(amount) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS running_max,
AVG(amount) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS running_avg,
STDDEV(amount) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS running_sd
FROM sales
ORDER BY sale_date;
```

4.4 RANGE vs ROWS — Critical Difference

ROWS vs RANGE — know the difference

```
-- ROWS: physical rows — always deterministic

SUM(amount) OVER (ORDER BY sale_date ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW)

-- RANGE: includes ALL rows with same ORDER BY value as current row
-- WARNING: if two rows share the same sale_date, RANGE includes both

SUM(amount) OVER (ORDER BY sale_date RANGE BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW)

-- Use ROWS for running totals (predictable, one row at a time)
-- Use RANGE for 'all rows on the same date' semantics
```

■ WARNING

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW is the correct frame for running totals. RANGE can produce surprising results when ORDER BY values tie.

Chapter 5

Pattern 3 — Lag, Lead & Row-Over-Row Comparisons

The LAG / LEAD Signatures

LAG / LEAD function signature

```
LAG (expression, offset, default) OVER (PARTITION BY ... ORDER BY ...)
LEAD (expression, offset, default) OVER (PARTITION BY ... ORDER BY ...)

-- offset : how many rows back (LAG) or forward (LEAD). Default = 1
-- default : value returned when the offset falls outside the partition
```

5.1 Day-over-Day Change

Day-over-day delta and % change per employee

```
SELECT
  sale_date,
  emp_id,
  amount,
  LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date) AS prev_amount,
  amount - LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date) AS delta,
  ROUND(
    100.0 * (amount - LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date))
    / NULLIF(LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date), 0),
    2) AS pct_change
FROM sales
ORDER BY emp_id, sale_date;
```

5.2 Multi-Period Comparison

Multi-period lag comparison

```
-- Compare current sale to 7 periods ago (same weekday last week)

SELECT
  sale_date,
```

```
amount,  
LAG(amount, 7, 0) OVER (PARTITION BY emp_id ORDER BY sale_date) AS week_ago,  
LAG(amount, 30, 0) OVER (PARTITION BY emp_id ORDER BY sale_date) AS month_ago,  
LEAD(amount, 1) OVER (PARTITION BY emp_id ORDER BY sale_date) AS next_period  
FROM sales  
ORDER BY emp_id, sale_date;
```

5.3 Detect Direction Changes (Peaks & Troughs)

Peak and trough detection

```
WITH flagged AS (  
  SELECT  
    recorded_at,  
    value,  
    LAG(value) OVER (PARTITION BY sensor_id ORDER BY recorded_at) AS prev_val,  
    LEAD(value) OVER (PARTITION BY sensor_id ORDER BY recorded_at) AS next_val  
  FROM sensor_readings  
  WHERE sensor_id = 42  
)  
SELECT  
  recorded_at,  
  value,  
  CASE  
    WHEN value > prev_val AND value > next_val THEN 'PEAK'  
    WHEN value < prev_val AND value < next_val THEN 'TROUGH'  
    ELSE 'NORMAL'  
  END AS signal_type  
FROM flagged  
WHERE prev_val IS NOT NULL AND next_val IS NOT NULL  
ORDER BY recorded_at;
```

5.4 First & Last Non-NULL Value (Forward Fill)

NULL forward-fill (LOCF — Last Observation Carried Forward)

```
-- Forward-fill NULL values with the last known non-NULL value
```

```
SELECT
  recorded_at,
  value,
  COALESCE(value,
    LAST_VALUE(value IGNORE NULLS)
  OVER (PARTITION BY sensor_id ORDER BY recorded_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
  ) AS filled_value
FROM sensor_readings
ORDER BY sensor_id, recorded_at;

-- PostgreSQL alternative (no IGNORE NULLS – use subquery)
SELECT recorded_at, value,
  MAX(value) OVER (PARTITION BY sensor_id,
    SUM(CASE WHEN value IS NULL THEN 0 ELSE 1 END)
  OVER (PARTITION BY sensor_id ORDER BY recorded_at)
  ) AS filled_value
FROM sensor_readings;
```

■ WARNING

IGNORE NULLS in LAST_VALUE / FIRST_VALUE is supported in Oracle, SQL Server 2022+, and MySQL 8.0.2+, but NOT in PostgreSQL. Use the SUM-based partition trick in PostgreSQL.

Chapter 6

Pattern 4 — Moving Averages & Rolling Windows

6.1 Simple Moving Average (SMA)

7-day simple moving average

```
-- 7-day simple moving average of daily sales

SELECT
  sale_date,
  SUM(amount) AS daily_total,
  ROUND(AVG(SUM(amount)) OVER (
    ORDER BY sale_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ), 2) AS sma_7d,
  COUNT(*) OVER (
    ORDER BY sale_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS window_size -- <7 at start of series
FROM sales
GROUP BY sale_date
ORDER BY sale_date;
```

6.2 Exponential Moving Average (EMA)

7-period EMA via recursive CTE

```
-- EMA cannot be expressed as a pure window function – needs recursion
--  $k = 2 / (N + 1)$ ,  $N = \text{period}$  (e.g. 7 days  $\rightarrow k = 0.25$ )

WITH RECURSIVE ema AS (
  -- Anchor: first row seeds EMA = its own value

  SELECT sale_date, daily_total, daily_total::DECIMAL(14,4) AS ema
  FROM (SELECT sale_date, SUM(amount) AS daily_total FROM sales
  GROUP BY sale_date ORDER BY sale_date LIMIT 1) first_day
```



```
UNION ALL

SELECT d.sale_date, d.daily_total,
ROUND(d.daily_total * 0.25 + ema.ema * 0.75, 4) -- k=0.25
FROM (SELECT sale_date, SUM(amount) AS daily_total FROM sales
GROUP BY sale_date) d
JOIN ema ON d.sale_date = (
SELECT MIN(s2.sale_date) FROM sales s2
WHERE s2.sale_date > ema.sale_date)
)
SELECT sale_date, daily_total, ema AS ema_7d
FROM ema ORDER BY sale_date;
```

6.3 Rolling Stddev & Bollinger Bands

Bollinger Bands — 20-day window

```
-- Bollinger Bands: SMA +/- 2 * rolling standard deviation

WITH daily AS (
SELECT sale_date, SUM(amount) AS daily_total FROM sales GROUP BY sale_date
),
bands AS (
SELECT
sale_date,
daily_total,
AVG(daily_total) OVER w AS sma_20,
STDDEV(daily_total) OVER w AS sd_20
FROM daily
WINDOW w AS (
ORDER BY sale_date
ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
)
)
SELECT
sale_date,
daily_total,
ROUND(sma_20, 2) AS middle_band,
```

```
ROUND(sma_20 + 2 * sd_20, 2) AS upper_band,  
ROUND(sma_20 - 2 * sd_20, 2) AS lower_band,  
CASE WHEN daily_total > sma_20 + 2 * sd_20 THEN 'OVERBOUGHT'  
WHEN daily_total < sma_20 - 2 * sd_20 THEN 'OVERSOLD'  
ELSE 'NORMAL' END AS signal  
FROM bands ORDER BY sale_date;
```

◆ TIP

The WINDOW alias ('WINDOW w AS (...)') avoids repeating the frame clause. Supported in PostgreSQL and MySQL 8+. SQL Server requires repeating the clause.

Chapter 7

Pattern 5 — Ranking & Percentile Functions

Function	Ties behaviour	Gaps after tie?	Use case
ROW_NUMBER()	Arbitrary order for ties	N/A	Unique sequential number, pagination
RANK()	Same rank for ties	Yes	Competition ranking (Olympic medals)
DENSE_RANK()	Same rank for ties	No	Ranking with no gaps (most common)
NTILE(n)	Splits into n equal buckets	N/A	Quartiles, deciles, percentile bands
PERCENT_RANK()	0.0 to 1.0 relative rank	N/A	What % of rows are below this value?
CUME_DIST()	Cumulative distribution 0..1	N/A	What % of rows are <= this value?

7.1 All Ranking Functions Side by Side

All ranking functions in one query

```
SELECT
  emp_id,
  dept,
  amount,
  ROW_NUMBER() OVER (PARTITION BY dept ORDER BY amount DESC) AS row_num,
  RANK() OVER (PARTITION BY dept ORDER BY amount DESC) AS rank,
  DENSE_RANK() OVER (PARTITION BY dept ORDER BY amount DESC) AS dense_rank,
  NTILE(4) OVER (PARTITION BY dept ORDER BY amount DESC) AS quartile,
  ROUND(PERCENT_RANK() OVER (PARTITION BY dept ORDER BY amount), 4) AS pct_rank,
  ROUND(CUME_DIST() OVER (PARTITION BY dept ORDER BY amount), 4) AS cume_dist
FROM sales
ORDER BY dept, amount DESC;
```

7.2 Top N Per Group (Efficient with ROW_NUMBER)

Top N per partition using ROW_NUMBER

```
-- Top 3 sales per department

WITH ranked AS (
  SELECT
    emp_id, dept, amount, sale_date,
    ROW_NUMBER() OVER (PARTITION BY dept ORDER BY amount DESC) AS rn
  FROM sales
)
SELECT emp_id, dept, amount, sale_date
FROM ranked
WHERE rn <= 3
ORDER BY dept, rn;
```

7.3 Percentile Values (Continuous & Discrete)

PERCENTILE_CONT and PERCENTILE_DISC

```
-- Median and percentile values per department

SELECT
  dept,
  PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY amount) AS median,
  PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY amount) AS p25,
  PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY amount) AS p75,
  PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY amount) AS p90,
  PERCENTILE_DISC(0.50) WITHIN GROUP (ORDER BY amount) AS median_disc
FROM sales
GROUP BY dept;

-- CONT: interpolates between rows (true mathematical percentile)
-- DISC: returns an actual existing value in the dataset
```

Chapter 8

Pattern 6 — Nth Value, First & Last in a Window

FIRST_VALUE / LAST_VALUE / NTH_VALUE — correct frame

```
SELECT
emp_id,
sale_date,
amount,
FIRST_VALUE(amount) OVER w AS first_sale,
LAST_VALUE(amount) OVER w AS last_sale, -- needs full frame!
NTH_VALUE(amount, 2) OVER w AS second_sale
FROM sales
WINDOW w AS (
PARTITION BY emp_id
ORDER BY sale_date
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
-- ^ MUST include UNBOUNDED FOLLOWING for LAST_VALUE to be correct
)
ORDER BY emp_id, sale_date;
```

■ WARNING

LAST_VALUE requires 'ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING'. Without it, the default frame only goes to the CURRENT ROW, making LAST_VALUE equal to the current row's value — a very common bug.

8.1 Compare Each Sale to Employee's Best & Worst

Individual performance vs personal best/worst

```
SELECT
emp_id,
sale_date,
amount,
```

```
MAX(amount) OVER (PARTITION BY emp_id) AS personal_best,  
MIN(amount) OVER (PARTITION BY emp_id) AS personal_worst,  
ROUND(100.0 * amount /  
NULLIF(MAX(amount) OVER (PARTITION BY emp_id), 0), 1  
 ) AS pct_of_best  
FROM sales  
ORDER BY emp_id, sale_date;
```

Chapter 9

Pattern 7 — Gap Detection & Missing Periods

■ The Gap Problem

In a time series, a gap is a period where data was expected but is absent. Gaps are invisible — no row means no row. You must generate the expected series and LEFT JOIN to reveal what is missing. This is the most important pattern in all of time-series SQL.

9.1 Detect Missing Dates in a Sale Series

Find all dates with no sales record

```
WITH RECURSIVE cal AS (  
  SELECT MIN(sale_date) AS dt FROM sales  
  
  UNION ALL  
  
  SELECT dt + INTERVAL '1 day' FROM cal  
  WHERE dt < (SELECT MAX(sale_date) FROM sales)  
)  
  
SELECT cal.dt AS missing_date  
FROM cal  
LEFT JOIN sales s ON s.sale_date = cal.dt  
WHERE s.sale_id IS NULL  
ORDER BY cal.dt;
```

9.2 Gap Between Consecutive Sensor Readings

Sensor reading gaps > 1 hour using LEAD

```
-- Find all gaps > 1 hour between consecutive readings (per sensor)  
  
WITH consecutive AS (  
  SELECT  
    sensor_id,  
    recorded_at AS curr_ts,  
    LEAD(recorded_at) OVER (PARTITION BY sensor_id
```

```
ORDER BY recorded_at) AS next_ts,
LEAD(recorded_at) OVER (PARTITION BY sensor_id
ORDER BY recorded_at)
- recorded_at AS gap_size
FROM sensor_readings
)
SELECT
sensor_id,
curr_ts AS gap_start,
next_ts AS gap_end,
gap_size AS gap_duration,
EXTRACT(EPOCH FROM gap_size) / 3600 AS gap_hours
FROM consecutive
WHERE gap_size > INTERVAL '1 hour'
AND next_ts IS NOT NULL
ORDER BY gap_hours DESC;
```

9.3 Largest Gap per Sensor

Largest gap per sensor (PostgreSQL DISTINCT ON)

```
WITH gaps AS (
SELECT
sensor_id,
recorded_at AS gap_start,
LEAD(recorded_at) OVER (PARTITION BY sensor_id ORDER BY recorded_at) AS gap_end,
LEAD(recorded_at) OVER (PARTITION BY sensor_id ORDER BY recorded_at)
- recorded_at AS gap_size
FROM sensor_readings
)
SELECT DISTINCT ON (sensor_id)
sensor_id,
gap_start,
gap_end,
gap_size AS max_gap
FROM gaps
```



```
WHERE gap_end IS NOT NULL  
ORDER BY sensor_id, gap_size DESC;
```

Chapter 10

Pattern 8 — Island Detection (Consecutive Groups)

■ The Islands Problem

An island is a maximal consecutive sequence of rows that belong together. The classic technique: assign each row a group number = ROW_NUMBER() minus another sequence that only increments when a new island starts. Rows in the same island share the same group number.

10.1 Consecutive Active Days per Employee

Gap & Island: consecutive active days

```
-- An 'island' = unbroken run of consecutive days with at least one sale

WITH daily_flags AS (
  SELECT DISTINCT emp_id, sale_date FROM sales
),
numbered AS (
  SELECT
    emp_id,
    sale_date,
    ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY sale_date) AS rn,
    sale_date - (ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY sale_date)
      * INTERVAL '1 day') AS grp
  -- grp is constant within a consecutive run
  FROM daily_flags
)
SELECT
  emp_id,
  MIN(sale_date) AS island_start,
  MAX(sale_date) AS island_end,
  COUNT(*) AS island_length_days
FROM numbered
```

```
GROUP BY emp_id, grp
ORDER BY emp_id, island_start;
```

10.2 Sensor Reading Islands (Value Above Threshold)

Islands where sensor value exceeds threshold

```
-- Island = consecutive readings where value > 100

WITH flagged AS (
  SELECT
    sensor_id,
    recorded_at,
    value,
    CASE WHEN value > 100 THEN 1 ELSE 0 END AS above_thresh
  FROM sensor_readings
),
grouped AS (
  SELECT
    sensor_id,
    recorded_at,
    value,
    above_thresh,
    ROW_NUMBER() OVER (PARTITION BY sensor_id ORDER BY recorded_at)
    - ROW_NUMBER() OVER (PARTITION BY sensor_id, above_thresh ORDER BY recorded_at)
  AS grp
  FROM flagged
)
SELECT
  sensor_id,
  MIN(recorded_at) AS island_start,
  MAX(recorded_at) AS island_end,
  COUNT(*) AS reading_count,
  AVG(value) AS avg_value_in_island
  FROM grouped
  WHERE above_thresh = 1
  GROUP BY sensor_id, grp
```

```
ORDER BY sensor_id, island_start;
```

10.3 Longest Island per Partition

Longest consecutive active streak per employee

```
WITH daily_flags AS (SELECT DISTINCT emp_id, sale_date FROM sales),
numbered AS (
  SELECT emp_id, sale_date,
  sale_date - (ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY sale_date)
  * INTERVAL '1 day') AS grp
FROM daily_flags
),
islands AS (
  SELECT emp_id, grp,
  MIN(sale_date) AS starts, MAX(sale_date) AS ends, COUNT(*) AS len
FROM numbered GROUP BY emp_id, grp
)
SELECT DISTINCT ON (emp_id)
emp_id, starts, ends, len AS longest_streak
FROM islands
ORDER BY emp_id, len DESC;
```

Chapter 11

Pattern 9 — Session & Event Windowing

■ Session Windowing

A session is a group of events from the same user/entity where the gap between consecutive events is smaller than a defined timeout (e.g. 30 minutes). Each time the gap exceeds the timeout, a new session begins. Use cases: web analytics, clickstream, sensor burst detection.

11.1 Assign Session IDs

Session detection with 30-minute timeout

```
WITH events AS (  
  SELECT  
    user_id,  
    event_time,  
    CASE WHEN event_time - LAG(event_time) OVER (  
      PARTITION BY user_id ORDER BY event_time)  
      > INTERVAL '30 minutes'  
    OR LAG(event_time) OVER (  
      PARTITION BY user_id ORDER BY event_time) IS NULL  
    THEN 1 ELSE 0  
  END AS is_new_session  
FROM user_events  
)  
,  
sessions AS (  
  SELECT  
    user_id,  
    event_time,  
    SUM(is_new_session) OVER (  
      PARTITION BY user_id ORDER BY event_time  
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS session_id
```

```
FROM events
)
SELECT
  user_id,
  session_id,
  MIN(event_time) AS session_start,
  MAX(event_time) AS session_end,
  COUNT(*) AS event_count,
  MAX(event_time) - MIN(event_time) AS session_duration
FROM sessions
GROUP BY user_id, session_id
ORDER BY user_id, session_start;
```

Chapter 12

Pattern 10 — Period Overlap & Allen's Interval Algebra

■ Allen's Interval Algebra

James Allen (1983) defined 13 mutually exclusive relations between two time intervals. The most practically useful: OVERLAP — interval A and B overlap if $A.start < B.end$ AND $A.end > B.start$. Memorise this as the two-predicate overlap test — it covers ALL overlap cases in one check.

Allen Relation	Condition	SQL Test
Before	A ends before B starts	$a.end_ts \leq b.start_ts$
After	A starts after B ends	$a.start_ts \geq b.end_ts$
Meets	A end = B start	$a.end_ts = b.start_ts$
Overlaps	A starts before B ends and ends after B starts	$a.start_ts < b.end_ts$ AND $a.end_ts > b.start_ts$
During	A is entirely inside B	$a.start_ts > b.start_ts$ AND $a.end_ts < b.end_ts$
Starts	A starts with B but ends before	$a.start_ts = b.start_ts$ AND $a.end_ts < b.end_ts$
Equals	A and B are identical intervals	$a.start_ts = b.start_ts$ AND $a.end_ts = b.end_ts$

12.1 Find Overlapping Bookings

All overlapping booking pairs with overlap duration

```
SELECT
  a.booking_id AS booking1,
  b.booking_id AS booking2,
  a.resource,
  GREATEST(a.start_ts, b.start_ts) AS overlap_start,
  LEAST(a.end_ts, b.end_ts) AS overlap_end,
  EXTRACT(EPOCH FROM
    LEAST(a.end_ts, b.end_ts) - GREATEST(a.start_ts, b.start_ts)
  ) / 3600.0 AS overlap_hours
FROM bookings a
```

```
JOIN bookings b
ON a.resource = b.resource
AND a.booking_id < b.booking_id -- unique pairs only
AND a.start_ts < b.end_ts -- Allen's overlap test
AND a.end_ts > b.start_ts
ORDER BY overlap_hours DESC;
```

12.2 Pack Overlapping Intervals into Merged Spans

Merge overlapping intervals into unified spans

```
-- Merge all overlapping bookings for a resource into continuous spans
WITH ordered AS (
  SELECT resource, start_ts, end_ts,
  MAX(end_ts) OVER (
    PARTITION BY resource
    ORDER BY start_ts
    ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
  ) AS prev_max_end
  FROM bookings
),
islands AS (
  SELECT resource, start_ts, end_ts,
  SUM(CASE WHEN start_ts > prev_max_end OR prev_max_end IS NULL
    THEN 1 ELSE 0 END)
  OVER (PARTITION BY resource ORDER BY start_ts) AS grp
  FROM ordered
)
SELECT resource,
MIN(start_ts) AS merged_start,
MAX(end_ts) AS merged_end
FROM islands
GROUP BY resource, grp
ORDER BY resource, merged_start;
```


Chapter 13

Pattern 11 — Calendar Tables & Business Day Arithmetic

13.1 Build a Calendar Table

Build a dim_date calendar table

```
CREATE TABLE dim_date AS
SELECT
  dt AS date_key,
  EXTRACT(YEAR FROM dt)::INT AS year,
  EXTRACT(MONTH FROM dt)::INT AS month,
  EXTRACT(DOW FROM dt)::INT AS dow, -- 0=Sun
  TO_CHAR(dt, 'Day') AS day_name,
  TO_CHAR(dt, 'Month') AS month_name,
  EXTRACT(WEEK FROM dt)::INT AS iso_week,
  EXTRACT(DOY FROM dt)::INT AS day_of_year,
  CASE WHEN EXTRACT(DOW FROM dt) IN (0,6)
  THEN FALSE ELSE TRUE END AS is_weekday,
  CASE WHEN EXTRACT(MONTH FROM dt) IN (3,4,5) THEN 'Q1'
  WHEN EXTRACT(MONTH FROM dt) IN (6,7,8) THEN 'Q2'
  WHEN EXTRACT(MONTH FROM dt) IN (9,10,11) THEN 'Q3'
  ELSE 'Q4' END AS fiscal_quarter,
  FALSE AS is_holiday -- populate separately
FROM generate_series('2020-01-01'::DATE, '2030-12-31'::DATE,
  INTERVAL '1 day') AS dt;

CREATE INDEX ON dim_date (date_key);
CREATE INDEX ON dim_date (is_weekday, is_holiday);
```

13.2 Business Days Between Two Dates

Business day counting and date arithmetic

```
-- Count working days (Mon-Fri, excluding holidays) between two dates
```

```
SELECT COUNT(*) AS business_days
FROM dim_date
WHERE date_key BETWEEN '2024-12-20' AND '2025-01-10'
AND is_weekday = TRUE
AND is_holiday = FALSE;

-- Add N business days to a date
SELECT date_key AS result_date
FROM dim_date
WHERE is_weekday = TRUE AND is_holiday = FALSE
AND date_key > '2024-12-20'
ORDER BY date_key
LIMIT 1 OFFSET 9; -- 10th business day after start
```

13.3 Sales Aggregated by Fiscal Period

Revenue by month with quarter-to-date running total

```
SELECT
d.year,
d.fiscal_quarter,
d.month,
SUM(s.amount) AS revenue,
COUNT(s.sale_id) AS transactions,
SUM(SUM(s.amount)) OVER (
PARTITION BY d.year, d.fiscal_quarter
ORDER BY d.month
ROWS UNBOUNDED PRECEDING
) AS qtd_revenue -- quarter-to-date
FROM sales s
JOIN dim_date d ON d.date_key = s.sale_date
GROUP BY d.year, d.fiscal_quarter, d.month
ORDER BY d.year, d.month;
```

Chapter 14

Pattern 12 — Time-Weighted Averages

■ Time-Weighted Average (TWA)

A simple average gives equal weight to each reading regardless of how long it was valid. A time-weighted average weights each reading by the duration it was in effect. Essential for: sensor monitoring, interest calculations, inventory valuation (VWAP), and any domain where readings can be spaced unevenly.

Time-weighted average using LEAD for duration

```
-- For each sensor, compute time-weighted average over a day

WITH readings_with_duration AS (
  SELECT
    sensor_id,
    recorded_at,
    value,
    LEAD(recorded_at) OVER (
      PARTITION BY sensor_id ORDER BY recorded_at
    ) AS next_ts,
    EXTRACT(EPOCH FROM (
      LEAD(recorded_at) OVER (
        PARTITION BY sensor_id ORDER BY recorded_at
      ) - recorded_at
    )) AS duration_secs
  FROM sensor_readings
  WHERE recorded_at::DATE = '2024-06-01'
)

SELECT
  sensor_id,
  ROUND(
    SUM(value * duration_secs) / NULLIF(SUM(duration_secs), 0),
    4) AS time_weighted_avg,
```

```
AVG(value) AS simple_avg, -- for comparison
SUM(duration_secs) / 3600.0 AS total_hours_covered
FROM readings_with_duration
WHERE duration_secs IS NOT NULL
GROUP BY sensor_id
ORDER BY sensor_id;
```

Chapter 15

Pattern 13 — Forecasting & Linear Regression in SQL

15.1 Linear Regression Using REGR_ Functions

Linear regression with REGR_ aggregate functions

```
-- Fit  $y = a + b \cdot x$  where  $x$  = day number,  $y$  = daily revenue

WITH daily AS (

SELECT
  sale_date,
  SUM(amount) AS revenue,
  (sale_date - MIN(sale_date) OVER ()):INT AS day_num
FROM sales
GROUP BY sale_date
)

SELECT
  REGR_SLOPE(revenue, day_num) AS slope,
  REGR_INTERCEPT(revenue, day_num) AS intercept,
  REGR_R2(revenue, day_num) AS r_squared,
  REGR_COUNT(revenue, day_num) AS n_points,
  -- Forecast next 7 days
  REGR_SLOPE(revenue, day_num) *
  (MAX(day_num) + 7) +
  REGR_INTERCEPT(revenue, day_num) AS forecast_7d_out
FROM daily;
```

15.2 Rolling Regression (Expanding Window)

Rolling 30-day linear regression slope

```
-- Compute slope and R2 over a 30-day rolling window

WITH daily AS (

SELECT sale_date, SUM(amount) AS revenue,
  ROW_NUMBER() OVER (ORDER BY sale_date) AS day_num
```

```
FROM sales GROUP BY sale_date
)
SELECT
sale_date,
revenue,
REGR_SLOPE(revenue, day_num) OVER (
ORDER BY sale_date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
) AS rolling_slope_30d,
REGR_R2(revenue, day_num) OVER (
ORDER BY sale_date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
) AS rolling_r2_30d
FROM daily
ORDER BY sale_date;
```

15.3 Year-over-Year Comparison

Year-over-year revenue comparison

```
WITH monthly AS (
SELECT
DATE_TRUNC('month', sale_date) AS month,
EXTRACT(YEAR FROM sale_date) AS yr,
EXTRACT(MONTH FROM sale_date) AS mo,
SUM(amount) AS revenue
FROM sales
GROUP BY 1,2,3
)
SELECT
curr.month,
curr.revenue AS current_revenue,
prev.revenue AS prior_year_revenue,
curr.revenue - prev.revenue AS yoy_delta,
ROUND(100.0 * (curr.revenue - prev.revenue)
/ NULLIF(prev.revenue, 0), 2) AS yoy_pct
FROM monthly curr
LEFT JOIN monthly prev
```

```
ON curr.yr = prev.yr + 1 AND curr.mo = prev.mo  
ORDER BY curr.month;
```

Chapter 16

Pattern 14 — Slowly Changing Dimensions (SCD Type 2)

■ SCD Type 2

Instead of overwriting historical data, SCD Type 2 inserts a new row for each change, timestamping when each version was valid. The current record has `valid_to = NULL` (or a far-future date). This preserves full history and enables point-in-time queries.

16.1 Point-in-Time Query — What Was the Employee's Salary On a Specific Date?

Point-in-time lookup on SCD Type 2 table

```
SELECT emp_id, dept, salary, valid_from, valid_to
FROM emp_history
WHERE emp_id = 42
AND valid_from <= '2023-06-15'
AND (valid_to > '2023-06-15' OR valid_to IS NULL)
ORDER BY valid_from;
```

16.2 Insert a New SCD2 Record (Upsert Pattern)

SCD Type 2 upsert — close old, open new

```
BEGIN;

-- Step 1: Close the current record
UPDATE emp_history
SET valid_to = CURRENT_DATE - INTERVAL '1 day',
is_current = FALSE
WHERE emp_id = 42
AND is_current = TRUE;

-- Step 2: Insert new version
INSERT INTO emp_history (emp_id, dept, salary, valid_from, valid_to, is_current)
VALUES (42, 'Engineering', 145000.00, CURRENT_DATE, NULL, TRUE);
```



```
COMMIT;
```

16.3 How Long Did Each Version Last?

Version duration and sequence number

```
SELECT
emp_id,
dept,
salary,
valid_from,
COALESCE(valid_to, CURRENT_DATE) AS effective_to,
COALESCE(valid_to, CURRENT_DATE) - valid_from AS days_active,
RANK() OVER (PARTITION BY emp_id ORDER BY valid_from) AS version_number
FROM emp_history
WHERE emp_id = 42
ORDER BY valid_from;
```

16.4 Current Salary vs Previous Salary

Salary raise history using LAG on SCD2 table

```
SELECT
emp_id,
salary AS current_salary,
LAG(salary) OVER (PARTITION BY emp_id ORDER BY valid_from) AS prev_salary,
salary - LAG(salary) OVER (PARTITION BY emp_id ORDER BY valid_from) AS
raise_amount
FROM emp_history
WHERE is_current = TRUE OR valid_to IS NOT NULL
ORDER BY emp_id, valid_from;
```

Chapter 17

Performance, Indexing & Anti-Patterns

Complexity by Pattern

Pattern	Complexity	Bottleneck	Optimisation
Date sequence gen	$O(N)$	CTE overhead	Use <code>generate_series()</code> in PostgreSQL
Running total	$O(N)$	Sort + scan	Index on (partition_col, order_col)
LAG / LEAD	$O(N)$	Sort pass	Index on (partition_col, order_col)
Moving average	$O(N)$	Frame computation	Index; avoid large RANGE frames
Gap detection	$O(N)$	Date join	Partial index on date column
Island detection	$O(N \log N)$	Double ROW_NUMBER	Materialise daily flags first
Session windowing	$O(N \log N)$	Sort + SUM window	Index on (user_id, event_time)
Overlap detection	$O(N^2)$	Self join	GiST range index (PostgreSQL)
Calendar join	$O(N)$	Join fanout	Pre-build dim_date, index date_key
SCD2 point-in-time	$O(1)$ – $O(k)$	Range predicate	Index on (emp_id, valid_from, valid_to)
Linear regression	$O(N)$	Aggregate pass	Materialise daily aggregates first

Indexing Checklist

- **Window function ORDER BY column** — Create index on (partition_col, order_col) — e.g. (emp_id, sale_date). This eliminates the sort step.
- **SCD2 valid_from / valid_to** — Composite index (emp_id, valid_from, valid_to) enables point-in-time lookups without full scans.
- **Overlap detection start/end** — PostgreSQL: GiST index on TSRANGE column. Other DBs: index start_ts + end_ts separately.
- **Calendar table date_key** — Primary key or unique index. Also index (is_weekday, is_holiday) for business day queries.
- **Sensor readings partition key** — (sensor_id, recorded_at) composite primary key — the most common query pattern.

Top Anti-Patterns

■ Using ORDER BY without PARTITION BY in window functions

- → Creates a single global window across the entire table — usually a bug. Always ask: 'Should this restart per user/sensor/dept?'

■ RANGE frame for running totals instead of ROWS

- → RANGE includes all peers with the same ORDER BY value, producing incorrect running totals when dates repeat.

■ Missing LAST_VALUE frame clause

- → Default frame ends at CURRENT ROW. LAST_VALUE without UNBOUNDED FOLLOWING always returns the current row's own value.

■ LEFT JOIN on calendar without indexing

- → calendar_table CROSS JOIN employee_table produces $N \times M$ rows — always materialise the per-employee calendar filtered to your date range first.

■ Recursive CTE for running totals

- → $O(N^2)$ vs $O(N)$ for window SUM(). Always prefer window functions for simple accumulation.

■ Self join for gap detection instead of LEAD

- → Self join is $O(N^2)$. LEAD/LAG is $O(N)$. Use window functions.

■ WHERE on window function result

- → Window functions cannot be filtered in the same SELECT's WHERE clause. Wrap in a CTE or subquery, then filter.

■ Overlapping SCD2 records

- → A missing or incorrect close on the old row creates two 'current' records. Always use a transaction and validate with: WHERE is_current = TRUE GROUP BY emp_id HAVING COUNT(*) > 1.

Chapter 18

Dialect Reference: PostgreSQL / MySQL / SQL Server / Oracle

Feature	PostgreSQL	MySQL 8+	SQL Server	Oracle
Date sequence	generate_series()	Recursive CTE	Recursive CTE / tally table	CONNECT BY LEVEL
Window functions	Full support	Full (8.0+)	Full support	Full support
RANGE INTERVAL	Supported	Supported	NOT supported	Supported
IGNORE NULLS	NOT supported	8.0.2+	2022+	Supported
WINDOW alias	Supported	Supported	NOT supported	NOT supported
PERCENTILE_CON T	Supported	NOT in window	Supported	Supported
REGR_ functions	Supported	Not supported	Not supported	Supported
GROUPS frame	11+	NOT supported	NOT supported	NOT supported
GiST range index	Supported	Not supported	Not supported	Not supported
LATERAL join	Supported	Supported	CROSS APPLY	Supported

Key Syntax Differences

Cross-dialect quick reference

```
-- Date arithmetic
-- PostgreSQL: date + INTERVAL '1 day'
-- MySQL: DATE_ADD(date, INTERVAL 1 DAY)
-- SQL Server: DATEADD(day, 1, date)
-- Oracle: date + 1

-- Recursive CTE keyword
-- PostgreSQL / MySQL: WITH RECURSIVE cte AS (...)
-- SQL Server / Oracle: WITH cte AS (...) -- no RECURSIVE keyword

-- String concatenation in paths
-- PostgreSQL / Oracle: ||
-- MySQL: CONCAT()
```

```
-- SQL Server: + or CONCAT()

-- DISTINCT ON (keep first row per group)
-- PostgreSQL: SELECT DISTINCT ON (col) ...
-- Others: Use ROW_NUMBER() subquery instead

-- Recursive max depth
-- SQL Server: OPTION (MAXRECURSION 500) -- default 100
-- Others: Add WHERE depth < N guard
```

Chapter 19

35 Graded Exercises

Three tiers — Beginner, Intermediate, Advanced. Use the schemas defined in Chapter 2. Work each tier completely before checking Chapter 20.

BEGINNER (Exercises 1 – 12)

Exercise 1

Generate all dates in Q1 2024 (Jan 1 – Mar 31) using a recursive CTE or `generate_series`.

Exercise 2

Show every day in January 2024 with total sales amount — including days with zero sales.

Exercise 3

Compute the running total of sales amount ordered by `sale_date` for all employees combined.

Exercise 4

For each sale, show the previous sale amount by the same employee using `LAG()`.

Exercise 5

Compute a 7-day simple moving average of daily total sales.

Exercise 6

Rank all employees by their total sales amount using `DENSE_RANK()`, highest first.

Exercise 7

Find all dates in the sales table where no sales were recorded (gaps in the series).

Exercise 8

Return the first and last sale date for each employee.

Exercise 9

For each sale, show the percentage it represents of that employee's total sales.

Exercise 10

Find the top 3 sales per department using `ROW_NUMBER()`.

Exercise 11

Compute the median sale amount per department using `PERCENTILE_CONT`.

Exercise 12

Using the emp_history SCD2 table, find each employee's current salary and department.

INTERMEDIATE (Exercises 13 – 25)

Exercise 13

Compute a per-employee running total that also shows the percentage of the employee's grand total achieved so far at each row.

Exercise 14

For each employee and each sale, show: current amount, previous amount, next amount, day-over-day change, and day-over-day percentage change.

Exercise 15

Compute a 30-day rolling mean AND 30-day rolling standard deviation of daily sales, and flag any day where the daily total is more than 2 standard deviations from the mean.

Exercise 16

Find all sensors that had a gap of more than 2 hours between any two consecutive readings.

Exercise 17

Find the longest consecutive streak of daily sales for each employee.

Exercise 18

Using the bookings table, find all overlapping booking pairs for the same resource, showing the overlap duration in hours.

Exercise 19

For each employee, show their salary change history using the SCD2 table: each version's salary, start date, end date, and salary delta from the previous version.

Exercise 20

Merge all overlapping sensor readings for sensor_id = 1 into non-overlapping continuous spans using the island technique.

Exercise 21

Build a monthly sales summary that shows: month, monthly total, month-over-month change, and a 3-month moving average.

Exercise 22

Assign session IDs to user events (table: user_events(user_id, event_time)) where a new session starts after 30 minutes of inactivity. Return session_id, start, end, duration, and event count per session.

Exercise 23

Compute the time-weighted average value for each sensor on each calendar day.

Exercise 24

For each day, compute the % change vs the same day last year (YoY) and the same day last month (MoM) for total sales.

Exercise 25

Using the SCD2 emp_history table, find the salary as of '2023-01-01' for every employee.

ADVANCED (Exercises 26 – 35)**Exercise 26**

Compute a 7-period Exponential Moving Average (EMA) of daily sales using a recursive CTE. $k = 2 / (7+1) = 0.25$.

Exercise 27

For the sensor_readings table, detect all 'anomalous' readings where the value is more than 3 standard deviations from the rolling 20-reading mean for that sensor. Return sensor_id, recorded_at, value, rolling_mean, rolling_sd, z_score.

Exercise 28

Implement Bollinger Bands on daily sales: 20-day SMA $\pm$ 2 standard deviations. Flag days as OVERBOUGHT, OVERSOLD, or NORMAL.

Exercise 29

Compute linear regression (slope, intercept, R^2) of daily sales over a rolling 30-day window and forecast the next day's sales based on the rolling model.

Exercise 30

Find all islands of consecutive days where total daily sales exceeded the overall daily average. For each island: start date, end date, length, and total revenue.

Exercise 31

For the bookings table, merge all overlapping intervals per resource into unified non-overlapping spans. Then compute total 'booked hours' per resource per day.

Exercise 32

Compute a complete SCD2 audit trail for each employee: show every salary version with duration (days active), rank in time, % salary change from previous version, and cumulative salary growth since the first record.

Exercise 33

Implement a VWAP (Volume-Weighted Average Price) calculation: given a trades table (trade_id, ticker, price, volume, trade_time), compute the VWAP for each ticker over rolling 1-hour windows.

Exercise 34

Create a complete calendar spine from Jan 2024 – Dec 2024 and produce a heatmap summary table: `week_number` × `day_of_week` → total sales. Weeks with no sales show 0.

Exercise 35

CHALLENGE: Write a single SQL statement (multiple CTEs allowed) that produces a complete time-series dashboard for each employee: `emp_id`, `sale_date`, `daily_amount`, 7-day SMA, 30-day SMA, `daily_rank_company_wide`, `daily_rank_in_dept`, `day_over_day_pct_change`, `is_personal_best` (boolean), `running_total`, `running_pct_of_annual_target` (assume target = 500000), and `island_id` (consecutive active day group). Handle all NULL edge cases.

Chapter 20

Full Answer Key

Answers 1 – 12 (Beginner)

Answer 1

```
WITH RECURSIVE cal AS (  
  SELECT CAST('2024-01-01' AS DATE) AS dt  
  UNION ALL  
  SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < '2024-03-31'  
)  
SELECT dt FROM cal ORDER BY dt;
```

Answer 2

```
WITH RECURSIVE cal AS (  
  SELECT CAST('2024-01-01' AS DATE) AS dt  
  UNION ALL  
  SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < '2024-01-31'  
)  
SELECT cal.dt, COALESCE(SUM(s.amount), 0) AS daily_sales  
FROM cal LEFT JOIN sales s ON s.sale_date = cal.dt  
GROUP BY cal.dt ORDER BY cal.dt;
```

Answer 3

```
SELECT sale_id, sale_date, amount,  
  SUM(amount) OVER (  
    ORDER BY sale_date, sale_id  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
  ) AS running_total  
FROM sales ORDER BY sale_date, sale_id;
```

Answer 4

```
SELECT emp_id, sale_date, amount,  
  LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date) AS prev_amount
```

```
FROM sales ORDER BY emp_id, sale_date;
```

Answer 5

```
WITH daily AS (SELECT sale_date, SUM(amount) AS total FROM sales GROUP BY
sale_date)

SELECT sale_date, total,
ROUND(AVG(total) OVER (
ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
), 2) AS sma_7d
FROM daily ORDER BY sale_date;
```

Answer 6

```
SELECT emp_id,
SUM(amount) AS total_sales,
DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS sales_rank
FROM sales
GROUP BY emp_id
ORDER BY sales_rank;
```

Answer 7

```
WITH RECURSIVE cal AS (
SELECT MIN(sale_date) AS dt FROM sales
UNION ALL
SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < (SELECT MAX(sale_date) FROM
sales)
)

SELECT cal.dt AS missing_date
FROM cal LEFT JOIN sales s ON s.sale_date = cal.dt
WHERE s.sale_id IS NULL ORDER BY cal.dt;
```

Answer 8

```
SELECT emp_id,
MIN(sale_date) AS first_sale,
MAX(sale_date) AS last_sale,
MAX(sale_date) - MIN(sale_date) AS tenure_days
```

```
FROM sales GROUP BY emp_id ORDER BY emp_id;
```

Answer 9

```
SELECT emp_id, sale_date, amount,  
ROUND(100.0 * amount / SUM(amount) OVER (PARTITION BY emp_id), 2) AS  
pct_of_total  
FROM sales ORDER BY emp_id, sale_date;
```

Answer 10

```
WITH ranked AS (  
SELECT emp_id, dept, amount, sale_date,  
ROW_NUMBER() OVER (PARTITION BY dept ORDER BY amount DESC) AS rn  
FROM sales  
)  
SELECT emp_id, dept, amount, sale_date  
FROM ranked WHERE rn <= 3 ORDER BY dept, rn;
```

Answer 11

```
SELECT dept,  
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY amount) AS median_sale  
FROM sales GROUP BY dept ORDER BY dept;
```

Answer 12

```
SELECT emp_id, dept, salary, valid_from  
FROM emp_history  
WHERE is_current = TRUE  
ORDER BY emp_id;
```

Answers 13 – 25 (Intermediate)

Answer 13

```
SELECT emp_id, sale_date, amount,  
SUM(amount) OVER (PARTITION BY emp_id ORDER BY sale_date  
ROWS UNBOUNDED PRECEDING) AS running_total,  
ROUND(100.0 *
```

```
SUM(amount) OVER (PARTITION BY emp_id ORDER BY sale_date ROWS UNBOUNDED
PRECEDING)

/ SUM(amount) OVER (PARTITION BY emp_id), 2) AS pct_progress

FROM sales ORDER BY emp_id, sale_date;
```

Answer 14

```
SELECT emp_id, sale_date, amount,

LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date) AS prev_amt,

LEAD(amount) OVER (PARTITION BY emp_id ORDER BY sale_date) AS next_amt,

amount - LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date) AS dod_delta,

ROUND(100.0 * (amount - LAG(amount) OVER (PARTITION BY emp_id ORDER BY
sale_date))

/ NULLIF(LAG(amount) OVER (PARTITION BY emp_id ORDER BY sale_date), 0), 2) AS
dod_pct

FROM sales ORDER BY emp_id, sale_date;
```

Answer 15

```
WITH daily AS (SELECT sale_date, SUM(amount) AS total FROM sales GROUP BY
sale_date),

stats AS (

SELECT sale_date, total,

AVG(total) OVER (ORDER BY sale_date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
AS mean_30,

STDDEV(total) OVER (ORDER BY sale_date ROWS BETWEEN 29 PRECEDING AND CURRENT
ROW) AS sd_30

FROM daily

)

SELECT sale_date, total, ROUND(mean_30,2), ROUND(sd_30,2),

ROUND((total - mean_30) / NULLIF(sd_30,0), 2) AS z_score,

CASE WHEN ABS(total - mean_30) > 2 * sd_30 THEN 'ANOMALY' ELSE 'OK' END AS flag

FROM stats ORDER BY sale_date;
```

Answer 16

```
WITH gaps AS (

SELECT sensor_id, recorded_at,

LEAD(recorded_at) OVER (PARTITION BY sensor_id ORDER BY recorded_at) AS next_ts
```

```
FROM sensor_readings
)
SELECT DISTINCT sensor_id
FROM gaps
WHERE next_ts - recorded_at > INTERVAL '2 hours'
ORDER BY sensor_id;
```

Answer 17

```
WITH flags AS (SELECT DISTINCT emp_id, sale_date FROM sales),
num AS (
SELECT emp_id, sale_date,
sale_date - (ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY sale_date)
* INTERVAL '1 day') AS grp
FROM flags
),
islands AS (
SELECT emp_id, grp, COUNT(*) AS streak
FROM num GROUP BY emp_id, grp
)
SELECT DISTINCT ON (emp_id) emp_id, MAX(streak) AS longest_streak
FROM islands GROUP BY emp_id ORDER BY emp_id, longest_streak DESC;
```

Answer 18

```
SELECT a.booking_id, b.booking_id AS overlaps_with, a.resource,
GREATEST(a.start_ts, b.start_ts) AS overlap_start,
LEAST(a.end_ts, b.end_ts) AS overlap_end,
EXTRACT(EPOCH FROM LEAST(a.end_ts,b.end_ts) -
GREATEST(a.start_ts,b.start_ts))/3600.0 AS hrs
FROM bookings a JOIN bookings b
ON a.resource = b.resource AND a.booking_id < b.booking_id
AND a.start_ts < b.end_ts AND a.end_ts > b.start_ts
ORDER BY hrs DESC;
```

Answer 19

```
SELECT emp_id, salary, valid_from, valid_to,
```

```
LAG(salary) OVER (PARTITION BY emp_id ORDER BY valid_from) AS prev_salary,  
salary - LAG(salary) OVER (PARTITION BY emp_id ORDER BY valid_from) AS raise,  
RANK() OVER (PARTITION BY emp_id ORDER BY valid_from) AS version  
FROM emp_history ORDER BY emp_id, valid_from;
```

Answer 20

```
WITH ordered AS (  
  SELECT recorded_at::DATE AS dt,  
  MAX(recorded_at::DATE) OVER (ORDER BY recorded_at::DATE  
    ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prev_max  
  FROM sensor_readings WHERE sensor_id = 1 GROUP BY recorded_at::DATE  
) ,  
islands AS (  
  SELECT dt,  
  SUM(CASE WHEN dt > prev_max OR prev_max IS NULL THEN 1 ELSE 0 END)  
  OVER (ORDER BY dt) AS grp  
  FROM ordered  
)  
SELECT MIN(dt) AS span_start, MAX(dt) AS span_end  
FROM islands GROUP BY grp ORDER BY span_start;
```

Answer 21

```
WITH monthly AS (  
  SELECT DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS rev  
  FROM sales GROUP BY 1  
)  
SELECT month, rev,  
rev - LAG(rev) OVER (ORDER BY month) AS mom_delta,  
ROUND(AVG(rev) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),  
2) AS ma_3m  
FROM monthly ORDER BY month;
```

Answer 22

```
WITH events AS (  
  SELECT user_id, event_time,
```

```

CASE WHEN event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY
event_time)
> INTERVAL '30 minutes' OR
LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
THEN 1 ELSE 0 END AS new_sess
FROM user_events
),
sessions AS (
SELECT user_id, event_time,
SUM(new_sess) OVER (PARTITION BY user_id ORDER BY event_time ROWS UNBOUNDED
PRECEDING) AS sess_id
FROM events
)
SELECT user_id, sess_id,
MIN(event_time) AS sess_start, MAX(event_time) AS sess_end,
COUNT(*) AS events, MAX(event_time) - MIN(event_time) AS duration
FROM sessions GROUP BY user_id, sess_id ORDER BY user_id, sess_start;

```

Answer 23

```

WITH durations AS (
SELECT sensor_id, recorded_at, value,
EXTRACT(EPOCH FROM LEAD(recorded_at) OVER (PARTITION BY sensor_id ORDER BY
recorded_at)
- recorded_at) AS secs
FROM sensor_readings
)
SELECT sensor_id, recorded_at::DATE AS day,
ROUND(SUM(value * secs) / NULLIF(SUM(secs), 0), 4) AS twa
FROM durations WHERE secs IS NOT NULL
GROUP BY sensor_id, recorded_at::DATE ORDER BY sensor_id, day;

```

Answer 24

```

WITH daily AS (
SELECT sale_date, SUM(amount) AS rev,
DATE_TRUNC('month', sale_date) AS mo

```



```

FROM sales GROUP BY sale_date
)
SELECT d.sale_date, d.rev,
d.rev - py.rev AS yoy_delta,
ROUND(100.0*(d.rev-py.rev)/NULLIF(py.rev,0),2) AS yoy_pct,
d.rev - pm.rev AS mom_delta,
ROUND(100.0*(d.rev-pm.rev)/NULLIF(pm.rev,0),2) AS mom_pct
FROM daily d
LEFT JOIN daily py ON py.sale_date = d.sale_date - INTERVAL '1 year'
LEFT JOIN daily pm ON pm.sale_date = d.sale_date - INTERVAL '1 month'
ORDER BY d.sale_date;

```

Answer 25

```

SELECT emp_id, dept, salary, valid_from, valid_to
FROM emp_history
WHERE valid_from <= '2023-01-01'
AND (valid_to > '2023-01-01' OR valid_to IS NULL)
ORDER BY emp_id;

```

Answers 26 – 35 (Advanced)

Answer 26

```

-- EMA: k=0.25, carry via recursive CTE
WITH RECURSIVE daily AS (
SELECT sale_date, SUM(amount) AS total FROM sales GROUP BY sale_date
),
ema AS (
SELECT sale_date, total, total::DECIMAL(14,4) AS ema_val
FROM daily ORDER BY sale_date LIMIT 1
UNION ALL
SELECT d.sale_date, d.total,
ROUND(d.total * 0.25 + e.ema_val * 0.75, 4)
FROM daily d JOIN ema e ON d.sale_date = (
SELECT MIN(sale_date) FROM daily WHERE sale_date > e.sale_date)
)

```

```
SELECT sale_date, total, ema_val AS ema_7p FROM ema ORDER BY sale_date;
```

Answer 27

```
WITH stats AS (
  SELECT sensor_id, recorded_at, value,
  AVG(value) OVER (PARTITION BY sensor_id ORDER BY recorded_at
  ROWS BETWEEN 19 PRECEDING AND CURRENT ROW) AS rolling_mean,
  STDDEV(value) OVER (PARTITION BY sensor_id ORDER BY recorded_at
  ROWS BETWEEN 19 PRECEDING AND CURRENT ROW) AS rolling_sd
  FROM sensor_readings
)
SELECT sensor_id, recorded_at, value,
ROUND(rolling_mean, 4) AS rolling_mean,
ROUND(rolling_sd, 4) AS rolling_sd,
ROUND((value - rolling_mean) / NULLIF(rolling_sd, 0), 3) AS z_score
FROM stats
WHERE ABS((value - rolling_mean) / NULLIF(rolling_sd, 0)) > 3
ORDER BY sensor_id, recorded_at;
```

Answer 28

```
WITH daily AS (SELECT sale_date, SUM(amount) AS total FROM sales GROUP BY
sale_date),
bands AS (
  SELECT sale_date, total,
  AVG(total) OVER (ORDER BY sale_date ROWS BETWEEN 19 PRECEDING AND CURRENT ROW)
  AS sma,
  STDDEV(total) OVER (ORDER BY sale_date ROWS BETWEEN 19 PRECEDING AND CURRENT
  ROW) AS sd
  FROM daily
)
SELECT sale_date, total,
ROUND(sma,2) AS middle, ROUND(sma+2*sd,2) AS upper, ROUND(sma-2*sd,2) AS lower,
CASE WHEN total > sma+2*sd THEN 'OVERBOUGHT'
WHEN total < sma-2*sd THEN 'OVERSOLD' ELSE 'NORMAL' END AS signal
FROM bands ORDER BY sale_date;
```

Answer 29

```
WITH daily AS (  
  SELECT sale_date, SUM(amount) AS total,  
  ROW_NUMBER() OVER (ORDER BY sale_date) AS day_num  
  FROM sales GROUP BY sale_date  
)  
  
SELECT sale_date, total,  
ROUND(REGR_SLOPE(total, day_num) OVER (ORDER BY sale_date ROWS BETWEEN 29  
PRECEDING AND CURRENT ROW), 2) AS slope,  
ROUND(REGR_R2(total, day_num) OVER (ORDER BY sale_date ROWS BETWEEN 29 PRECEDING  
AND CURRENT ROW), 4) AS r2,  
ROUND(  
  REGR_SLOPE(total, day_num) OVER (ORDER BY sale_date ROWS BETWEEN 29 PRECEDING  
AND CURRENT ROW)  
  * (day_num + 1) +  
  REGR_INTERCEPT(total, day_num) OVER (ORDER BY sale_date ROWS BETWEEN 29  
PRECEDING AND CURRENT ROW)  
  , 2) AS next_day_forecast  
FROM daily ORDER BY sale_date;
```

Answer 30

```
WITH daily AS (SELECT sale_date, SUM(amount) AS rev FROM sales GROUP BY  
sale_date),  
avg_rev AS (SELECT AVG(rev) AS overall_avg FROM daily),  
flagged AS (  
  SELECT d.sale_date, d.rev,  
  CASE WHEN d.rev > a.overall_avg THEN 1 ELSE 0 END AS above  
  FROM daily d CROSS JOIN avg_rev a  
)  
grouped AS (  
  SELECT sale_date, rev, above,  
  ROW_NUMBER() OVER (ORDER BY sale_date)  
  - ROW_NUMBER() OVER (PARTITION BY above ORDER BY sale_date) AS grp  
  FROM flagged  
)  
  
SELECT MIN(sale_date) AS island_start, MAX(sale_date) AS island_end,
```

```
COUNT(*) AS days, SUM(rev) AS total_revenue
FROM grouped WHERE above = 1
GROUP BY grp ORDER BY island_start;
```

Answer 31

```
WITH ordered AS (
  SELECT resource, start_ts, end_ts,
  MAX(end_ts) OVER (PARTITION BY resource ORDER BY start_ts
  ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prev_max
  FROM bookings
),
islands AS (
  SELECT resource, start_ts, end_ts,
  SUM(CASE WHEN start_ts > prev_max OR prev_max IS NULL THEN 1 ELSE 0 END)
  OVER (PARTITION BY resource ORDER BY start_ts) AS grp
  FROM ordered
),
merged AS (
  SELECT resource, grp, MIN(start_ts) AS span_start, MAX(end_ts) AS span_end
  FROM islands GROUP BY resource, grp
)
SELECT resource, span_start::DATE AS day,
SUM(EXTRACT(EPOCH FROM LEAST(span_end, (span_start::DATE+1)::TIMESTAMP)
- GREATEST(span_start, span_start::DATE::TIMESTAMP))) / 3600.0 AS booked_hours
FROM merged GROUP BY resource, span_start::DATE ORDER BY resource, day;
```

Answer 32

```
SELECT emp_id, salary, valid_from, valid_to,
COALESCE(valid_to, CURRENT_DATE) - valid_from AS days_active,
RANK() OVER (PARTITION BY emp_id ORDER BY valid_from) AS version,
salary - LAG(salary) OVER (PARTITION BY emp_id ORDER BY valid_from) AS raise,
ROUND(100.0 * (salary - LAG(salary) OVER (PARTITION BY emp_id ORDER BY
valid_from))
/ NULLIF(LAG(salary) OVER (PARTITION BY emp_id ORDER BY valid_from), 0), 2) AS
raise_pct,
```

```

ROUND(100.0 * (salary - FIRST_VALUE(salary) OVER (PARTITION BY emp_id ORDER BY
valid_from

ROWS UNBOUNDED PRECEDING))

/ NULLIF(FIRST_VALUE(salary) OVER (PARTITION BY emp_id ORDER BY valid_from

ROWS UNBOUNDED PRECEDING), 0), 2) AS cumulative_growth_pct

FROM emp_history ORDER BY emp_id, valid_from;

```

Answer 33

```

-- VWAP = SUM(price * volume) / SUM(volume) over rolling 1-hour window

SELECT ticker, trade_time, price, volume,

ROUND(

SUM(price * volume) OVER (

PARTITION BY ticker

ORDER BY trade_time

RANGE BETWEEN INTERVAL '1 hour' PRECEDING AND CURRENT ROW

) / NULLIF(

SUM(volume) OVER (

PARTITION BY ticker

ORDER BY trade_time

RANGE BETWEEN INTERVAL '1 hour' PRECEDING AND CURRENT ROW

), 0), 4) AS vwap_1h

FROM trades ORDER BY ticker, trade_time;

```

Answer 34

```

WITH RECURSIVE cal AS (

SELECT CAST('2024-01-01' AS DATE) AS dt

UNION ALL SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < '2024-12-31'

),

annotated AS (

SELECT dt,

EXTRACT(WEEK FROM dt)::INT AS wk,

EXTRACT(DOW FROM dt)::INT AS dow

FROM cal

),

daily_sales AS (

```

```

SELECT sale_date, SUM(amount) AS rev FROM sales GROUP BY sale_date
)

SELECT a.wk AS week_number, a.dow AS day_of_week,
COALESCE(ds.rev, 0) AS total_sales
FROM annotated a
LEFT JOIN daily_sales ds ON ds.sale_date = a.dt
ORDER BY a.wk, a.dow;

```

Answer 35

```

-- CHALLENGE: Full per-employee time-series dashboard

WITH daily_flags AS (SELECT DISTINCT emp_id, sale_date FROM sales),
numbered AS (
SELECT emp_id, sale_date,
sale_date - (ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY sale_date)
* INTERVAL '1 day') AS island_grp
FROM daily_flags
),
island_ids AS (
SELECT emp_id, sale_date,
DENSE_RANK() OVER (PARTITION BY emp_id ORDER BY island_grp) AS island_id
FROM numbered
),
daily AS (
SELECT s.emp_id, s.sale_date, SUM(s.amount) AS daily_amt
FROM sales s GROUP BY s.emp_id, s.sale_date
)
SELECT
d.emp_id,
d.sale_date,
d.daily_amt,
ROUND(AVG(d.daily_amt) OVER (PARTITION BY d.emp_id ORDER BY d.sale_date
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 2) AS sma_7d,
ROUND(AVG(d.daily_amt) OVER (PARTITION BY d.emp_id ORDER BY d.sale_date
ROWS BETWEEN 29 PRECEDING AND CURRENT ROW), 2) AS sma_30d,

```

```
RANK() OVER (PARTITION BY d.sale_date ORDER BY d.daily_amt DESC) AS
daily_rank_company,
RANK() OVER (PARTITION BY d.sale_date, s.dept ORDER BY d.daily_amt DESC) AS
daily_rank_dept,
ROUND(100.0*(d.daily_amt - LAG(d.daily_amt) OVER (PARTITION BY d.emp_id ORDER BY
d.sale_date))
/ NULLIF(LAG(d.daily_amt) OVER (PARTITION BY d.emp_id ORDER BY
d.sale_date),0),2) AS dod_pct,
d.daily_amt = MAX(d.daily_amt) OVER (PARTITION BY d.emp_id) AS is_personal_best,
SUM(d.daily_amt) OVER (PARTITION BY d.emp_id ORDER BY d.sale_date ROWS UNBOUNDED
PRECEDING) AS running_total,
ROUND(100.0 * SUM(d.daily_amt) OVER (PARTITION BY d.emp_id ORDER BY d.sale_date
ROWS UNBOUNDED PRECEDING) / 500000.0, 2) AS pct_of_target,
ii.island_id
FROM daily d
JOIN sales s ON s.emp_id = d.emp_id AND s.sale_date = d.sale_date
JOIN island_ids ii ON ii.emp_id = d.emp_id AND ii.sale_date = d.sale_date
ORDER BY d.emp_id, d.sale_date;
```

— End of Guide —

Time is the most complex dimension in data. Master these 14 patterns and you will solve 95% of all time-series problems you encounter in production systems. Practice every exercise without looking at answers first — that discomfort is where the learning happens.